CPT102 Data Structures: Lecture 6: Faster sorting

Thomas Selig

XJTLU

April 6, 2026

Reminders/announcements

CW Project announcements

- Next Monday is deadline for take-home submission.
- ICT will be **Wednesday, 15 April, at 19:00**.
  - Duration is 45 minutes
  - Check your room in spreadsheet on Learning Mall.
  - Learning Mall announcement for modalities, including timings, SEB usage, authorised materials, etc.
- ICT questions:
  - Six short-answer or MCQs;
  - Two more involved analytical questions (still under preparation, but likely gap-fill).

  Any coding questions will be in Java, since solution requires Java. No pseudocode!
- More detail on ICT and some feedback on take-home project will be provided in next week's lectures.

Contents of today's lecture

More sorting, with faster algorithms!

- General sorting: quick sort and merge sort
- Complexity analysis in worst- and average-cases.
  - Big-Theta and Big-Omega notations;
  - Master theorem.
- Even faster sorting on structured input: counting sort and radix sort.

Lecture Learning Outcomes

By the end of today's lecture, you should be able to:

A. Explain fast general sorting algorithms, including merge sort and quick sort, and implement them in pseudocode.

B. Describe counting sort and radix sort algorithms, explaining how they utilise structured data to perform faster than general sorting algorithms.

C. Apply the master theorem to analyse the complexity of sorting algorithms.

Reminder: last week

- Last week, we introduced **insertion sort** and **selection sort**.
- Both algorithms partition the array into a sorted part (left) and unsorted part (right).
- Each step moves a single element from the unsorted part into the sorted part.
  - Insertion sort: take first (by index) unsorted element, *insert* into sorted part.
  - Selection sort: *select* minimum remaining element in unsorted part, place at tail of sorted part.
- Repeat step until the whole array is sorted.

Recursive implementations

Algorithm 1: recursive insertion sort

Procedure RecursiveInsertionSort(`arr`, `fromIndex`)

▷ sorted part is `arr[0:fromIndex]`

`nextSortedItem` ← `arr[fromIndex]`;

Insert(`nextSortedItem`, `arr`, `fromIndex`);

▷ insert `nextSortedItem` into correct place in sorted part

RecursiveInsertionSort(`arr`, `fromIndex` + 1);

Algorithm 2: recursive selection sort

Procedure RecursiveSelectionSort(`arr`, `fromIndex`)

▷ sorted part is `arr[0:fromIndex]`

`minRemainingIndex` ← SelectMinIndex(`arr`, `fromIndex`);

▷ select index of minimum remaining item in unsorted part

Swap(`arr`, `fromIndex`, `minRemainingIndex`);

RecursiveSelectionSort(`arr`, `fromIndex` + 1);

Towards faster sorting (1/2)

- Previous slide clearly shows the recurrence relation: $T(n) = T(n-1) + f(n)$, where $f(n)$ is the additional cost of the recursive step.
  - Insertion sort: step takes next remaining element, inserts it into sorted array; cost is **insertion** operation (call to Insert).
  - Selection sort: step selects (index of) minimal remaining element, swaps it with the first unsorted element; cost is **selection** of remaining min (call to SelectMinIndex).
- Both of these steps have additional **linear** time complexity, so in total we get $T(n) = T(n-1) + \mathrm{O}(n)$, giving $\boxed{T(n) = \mathrm{O}\left(n^2\right)}$.

Question

Can we do better?

Towards faster sorting (2/2)

- Recall from last week linear search vs binary search:
  - Linear search: $T(n) = T(n-1) + \mathrm{O}(1)$ gives $T(n) = \mathrm{O}(n)$.
  - Binary search: $T(n) = T\left(\frac{n}{2}\right) + \mathrm{O}(1)$ gives $T(n) = \mathrm{O}(\log(n))$.
- Can we hope for $T(n) = T\left(\frac{n}{2}\right) + f(n)$ for sorting algorithms?
- Not really: would mean only need to sort half the array with some re-assembly operations, which seems optimistic.
- However, we can hope for $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + f(n)$.
  - Intuitively, break array into two equal parts, sort both parts, re-assemble.
  - Known as **divide-and-conquer** approach.

Merge sort: overview

The idea behind merge sort is simple.

1. **Divide**: we break the array into two sub-arrays of (roughly) equal length.
2. **Conquer**: recursively sort each sub-array.
3. **Combine**: merge the two sorted halves into a single sorted array.

Example: input array is [5, 1, 2, 6, 4, 3].

1. **Divide**: sub-arrays are [5, 1, 2] (left) and [6, 4, 3] (right).
2. **Conquer**: recursively sort sub-arrays to get [1, 2, 5] (left) and [3, 4, 6] (right).
3. **Combine**: merge the two sorted halves to get [1, 2, 3, 4, 5, 6].

Merge sort: towards implementation

- As with most divide-and-conquer algorithms, standard implementation of merge sort is recursive.
  - This means we will use helper methods with additional parameters `start` and `end`.
- Unlike last week's algorithms (and quick sort), the standard implementation of merge sort is not in-place.
  - Instead, in the "Combine" step, merge sort requires an auxiliary array of the same length as the array to be sorted.
  - To optimise memory usage, the auxiliary array will also be passed into the helper method.

Merge sort: implementation

Algorithm 3: merge sort

Procedure MergeSort(`T[] arr`)

```
temp ← new T[arr.length];
MergeSortHelper(arr, temp, 0, arr.length);
```

Procedure MergeSortHelper(`arr, temp, start, end`)

```
if end - start ≤ 1 then
    return ;        ▷ If the sub-array has length at most 1, nothing to do
mid ← (start + end) / 2;
MergeSortHelper(arr, temp, start, mid);
MergeSortHelper(arr, temp, mid, end);
Merge(arr, temp, start, mid, end);
```

- Obviously incomplete: need to implement Merge procedure.
- Still useful: we've *decomposed* the problem, and *reduced* it to the Merge problem.

The merge operation: implementation

Algorithm 4: merging two sorted arrays

Input: An array `arr` and indices `start`, `mid`, `end` such that `arr[start:mid]` and `arr[mid:end]` are sorted

Output: The modified array `arr` such that `arr[start:end]` is sorted

```
COPY(arr, temp, start, end);      ▷ copy the sub-array arr[start:end] into temp
i ← start, j ← mid, k ← start;
▷   indices for left sub-array, right sub-array, merged result
while i < mid and j < end do
    if LESS(temp[j], temp[i]) then
        arr[k] ← temp[j], j ← j + 1;
    else
        arr[k] ← temp[i], i ← i + 1;
    k ← k + 1;
while i < mid do
    arr[k] ← temp[i], i ← i + 1, k ← k + 1;
    ▷   copy remaining elements from left-half, if any
▷   no need to copy remaining elements from right-half, as already in correct place
```

This implementation ensures that merge sort is **stable**.

Merge sort: time complexity analysis

- Clearly, we have $T(n) = 2 \cdot T\left(\frac{n}{2}\right) + T'(n)$, where $T'(n)$ is the complexity of MERGE(`arr`, `temp`, 0, $n/2$, n).
- Complexity of MERGE?
  - Copy elements from `start` to `end` n assignments.
  - Compare `temp`[i] and `temp`[j], incrementing one of i or j over ranges at most $n/2$ for each $\Rightarrow$ at most n comparisons.
  - At most n assignments to the array `arr`.

  Total complexity is therefore $T'(n) = \mathrm{O}(n)$.
- This gives $\boxed{T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathrm{O}(n)}$

Complexity analysis: lower bounds and Big-Theta

- Previous slide gave us $\boxed{T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \mathrm{O}(n)}$
- Can "show" (hand-waving) that this gives $T(n) = \mathrm{O}(n \log(n))$.
- More formally, this is a special case of the **master theorem** of algorithm analysis.
- Recall that $f(n) = \mathrm{O}(g(n))$ means $\boxed{f(n) \leq C \cdot g(n)}$ (upper bound).

Definition (lower bound)

We say that $f(n)$ is a **Big-Omega** of $g(n)$, and write $f(n) = \Omega(g(n))$ if $g(n) = \mathrm{O}(f(n))$, equivalently $\boxed{f(n) \geq c \cdot g(n)}$.
We say that $f(n)$ is a **Big-Theta** of $g(n)$, and write $f(n) = \Theta(g(n))$ if $f(n)$ is both a Big-Oh and a Big-Omega of $g(n)$, equivalently $\boxed{c \cdot g(n) \leq f(n) \leq C \cdot g(n)}$.

Complexity analysis: master theorem

Master theorem

Suppose that we have a recurrence relation of the form

$$\boxed{T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)}$$

We define the critical exponent $c_{\text{crit}} := \log_b(a) = \frac{\log(a)}{\log(b)}$.

Case 1 If $f(n) = \mathrm{O}\left(n^c\right)$ for some $c < c_{\text{crit}}$, then we have

$$\boxed{T(n) = \Theta\left(n^{c_{\text{crit}}}\right)}$$

Case 2 If $f(n) = \Theta\left(n^{c_{\text{crit}}} \cdot (\log(n))^k\right)$ for some $k \geq 0$, then we have

$$\boxed{T(n) = \Theta\left(n^{c_{\text{crit}}} \cdot (\log(n))^{k+1}\right)}$$

Case 3 If $f(n) = \Omega\left(n^c\right)$ for some $c > c_{\text{crit}}$, and some additional "regularity" condition holds on f, then we have

$$\boxed{T(n) = \Theta\left(f(n)\right)}$$

Master theorem interpretation

- In the recurrence relation, a is the number of sub-problems to solve, and n/b the size of each sub-problem.
  - The critical exponent therefore measures the (logarithmic) ratio of these two parameters. Intuitively this relates to the size of the recursive tree.

  $f(n)$ measures the re-assembly cost, sometimes called *splitting term*.

- How to interpret each case:

  Case 1 $f(n) = \mathrm{O}\left(n^c\right)$ for $c < c_{\mathrm{crit}}$: the splitting term is small, the recursive tree dominates.

  Case 2 The recursive tree and splitting term are "balanced", both appear in the final complexity.

  Case 3 $f(n) = \Omega\left(n^c\right)$ for $c > c_{\mathrm{crit}}$: the splitting term dominates.

Back to merge sort: complexity

Time complexity

- Recall that $\boxed{T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \Theta(n)}$.
- Here we have $a = b$, so $c_{\text{crit}} = 1$. Therefore we are in Case 2 with $k = 0$.
- Master theorem gives $\boxed{T(n) = \Theta(n \log(n))}$ (log-linear).
- Compares very favourably to last week's algorithms which had $T(n) = \Theta(n^2)$.

Space complexity

- Need an auxiliary array of size n.
  - Array shared by all recursive calls: only need $\mathrm{O}(n)$ space in total!
- Size of call stack is depth of the recursive tree, so $\mathrm{O}(\log(n))$.
- The auxiliary array dominates, so $\boxed{M(n) = \Theta(n)}$ (linear).

Quick sort: overview

Quick sort is another divide-and-conquer algorithm, which works by selecting a pivot element and partitioning the array around it.

1. **Choose a pivot**: select an element of the array (various strategies exist)
   - In this course, assume SelectPivotIndex procedure that selects the index of a "good" pivot.
2. **Partition the array**: re-arrange elements of the array so that all the elements with value less than the pivot are placed to the pivot's left, and all the elements with value greater than the pivot are placed to the pivot's right.
   - Relative order of elements in left and right sub-arrays is unchanged.
3. **Recursively sort**: apply the same process to the left and right sub-arrays.

Partition step: example

- Start with the array [5, 9, 1, 3, 7].
- Select a pivot, say 7.
  - Elements < 7 go to its left;
  - Elements > 7 go to its right;
- After applying this, we get:

$$\underbrace{[5,\ 1,\ 3]}_{\text{left}} \quad \underbrace{|\ 7\ |}_{\text{pivot}} \quad \underbrace{[9]}_{\text{right}}$$

- We then repeat on the left and right sub-arrays (on the right there is nothing to do here).

Key observation: After the partition step, the pivot element is in the correct position in the array!

Merge sort: implementation

Algorithm 5: quick sort

Procedure QuickSort(`T[] arr`)
QuickSortHelper(`arr`, 0, `arr.length`);

Procedure QuickSortHelper(`arr`, `start`, `end`)
if *end - start* $\leq$ *1* **then**
  return ; ▷ If the sub-array has length at most 1, nothing to do
`pivotIndex` $\leftarrow$ SelectPivotIndex(`arr`, `start`, `end`);
▷ we have `start` $\leq$ `pivotIndex` $<$ `end`
`pivotIndex` $\leftarrow$ Partition(`arr`, `start`, `end`, `pivotIndex`);
QuickSortHelper(`arr`, `start`, `pivotIndex`);
QuickSortHelper(`arr`, `pivotIndex` + 1, `end`);

- Similar to merge sort, key step is Partition procedure.

The partition step: implementation

In fact, various implementations exist. Here is an example.

Algorithm 6: the partition step in quick sort

Procedure PARTITION(`arr`, `start`, `end`, `pivotIndex`)

`pivot` $\leftarrow$ `arr[pivotIndex]`, $i \leftarrow$ `start`;

SWAP(`arr`, `pivotIndex`, `end` - 1); ▷ moves the pivot out of the way

for $j \leftarrow$ *start* **to** *end* - 2 **do**

  if LESS(*arr[j], pivot*) **then**

    SWAP(`arr`, i, j);

    $i \leftarrow i + 1$;

SWAP(`arr`, i, `end` - 1); ▷ put the pivot back in the right place

return i;

- Partitions the sub-array `arr[start:end]` around the pivot, and returns the pivot's index in the partitioned array.
- Standard implementation uses "$\leq$" for the comparison.

Quick sort: complexity analysis (1/2)

- Partitioning step is $\mathrm{O}(n)$ since it traverses the (sub-)array once.
- Recurrence relation is therefore: $T(n) = T(k) + T(n-1-k) + \mathrm{O}(n)$, where k is the pivot index returned by the partition step.
- If pivot is poorly chosen, e.g. to be the maximal element in the array, this just gives $T(n) = T(n-1) + \mathrm{O}(n)$.
  - In this case, we just get $\boxed{T_{\text{worst}}(n) = \mathrm{O}(n^2)}$
  - No better than insertion/selection sort!
- However, good pivot selection strategies exist.
  - For example, sample 3 elements randomly, and choose the median value.
  - For such cases, we can show that $k \approx \frac{n}{2}$, which gives $T(n) \approx 2 \cdot T\left(\frac{n}{2}\right) + \mathrm{O}(n)$. Sames as merge sort!
  - This gives $\boxed{T_{\text{best}}(n) = T_{\text{avg}}(n) = \mathrm{O}(n \log(n))}$

Quick sort: complexity analysis (2/2)

Time complexity – summary

- For poor pivot choices, we get $\boxed{T_{\text{worst}}(n) = \mathrm{O}\left(n^2\right)}$
- For good pivot choices, we get $\boxed{T_{\text{best}}(n) = T_{\text{avg}}(n) = \mathrm{O}\left(n \log(n)\right)}$
- In general, quick sort is quicker than merge sort due to smaller multiplicative constants and lower memory overhead.

Space complexity

- Quick sort is **in place**: no need for an auxiliary array.
- Memory complexity is depth of the recursion tree. Again, depends on having good pivot choices.
  - For poor choices, we get $\boxed{M_{\text{worst}}(n) = \mathrm{O}\left(n\right)}$
  - For good choices, we get $\boxed{M_{\text{best}}(n) = M_{\text{avg}}(n) = \mathrm{O}\left(\log(n)\right)}$

Towards even faster sorting

Question

Can we do better than the $\mathrm{O}(n \log(n))$ time complexity?

- In general, no!
  - These is a theorem that proves that any general sorting algorithm has time complexity $\Omega(n \log(n))$.
  - This gives a theoretical lower bound on the performance of general sorting algorithms.
- However, this only applies to *general* sorting algorithms, which use only pairwise comparisons of elements and assignments.
- If we put additional conditions on the input array, we can in fact do better.
  - Examples: counting sort, radix sort.

  Both of these assume arrays have a limited "range" of values

Counting sort: overview

- Counting sort is a **non-comparison sorting algorithm** which assumes the input array `arr` values are integers in the range $[0, m)$.
  - Easily extends to an array with a finite set of (known) values.
- How it works:
  - Use an auxiliary array `count` of length m.
  - For each $i \in [0, m)$, `count`$[i]$ will be the number of occurrences of i in `arr`.
  - Use this to recover the original sorted array.
- It runs in linear $\mathrm{O}(m + n)$ time, fast!

Example: [0, 1, 0, 3, 3, 1, 2, 1]

Counting sort: implementation

Algorithm 7: counting sort

Input: An array with values in the range $\{0, \ldots, m-1\}$
Output: The sorted array
Procedure CountingSort(`arr`)

```
count ← new int[m];                    ▷ auxiliary array, initialised to all 0's
foreach item in arr do
    count[item]++;                     ▷ count how many times item appears in arr
index ← 0;
for value ← 0 to m − 1 do
    while count[value] > 0 do
        arr[index] ← value, index++;
        count[value]--;
```

Counting sort: complexity analysis

Time complexity

- Initialising new array is $\mathrm{O}(m)$.
- First loop is simple loop over `arr` $\Rightarrow \mathrm{O}(n)$
- Second loop:
  - Notice that inside the while loop, we always increment the index.
  - At most n such incrementations, so the *cumulative* number of operations for the while loop is $\mathrm{O}(n)$.
  - There are also m iterations of the for loop, giving $\mathrm{O}(m)$.

Therefore, total time complexity is $\boxed{T(n) = \mathrm{O}(m+n)}$

- Faster than comparison-based sorting algorithms when m is relatively small, e.g. $m = \mathrm{O}(n)$.

Space complexity

- Auxiliary array of length m, so space complexity is $\boxed{M(n) = \mathrm{O}(m)}$

Radix sort: overview

Our final (yes, really!) sorting algorithm: **radix sort**.

- Like counting sort, non-comparison based. Assumes inputs are all non-negative integers with (at most) d decimal digits.
- Simply repeats a *stable* sorting algorithm by digits, from the least significant to the most significant.
- Because there are only 10 possible values for each digit, can use counting sort for each pass!

Example: input = [170, 45, 75, 90, 802, 44, 2, 66]

1. Sort by least significant digit (units or 1's). Keep relative order of elements which have the same units.
   This gives [170, 090, 802, 002, 044, 045, 075, 066]
2. Sort by 10's digit, keeping relative order of elements with same digit.
   This gives [802, 002, 044, 045, 066, 170, 075, 090]
   - Because of stability, last two digits are properly ordered!
3. Finally, sort by 100's, again maintaining stability. This gives the final sorted array [002, 044, 045, 066, 075, 090, 170, 802].

Radix sort: partial implementation

Algorithm 8: radix sort

Procedure RadixSort(`arr`)
Input: An array of non-negative integers, each with at most d digits
Output: The array, sorted
for $i \leftarrow 0$ **to** $d - 1$ **do**
    CountSortByDigit(`arr`, i);

Procedure CountSortByDigit(`arr`, i)
`count` $\leftarrow$ `new int`[10];
foreach *item* **in** *arr* **do**
    `digit` $\leftarrow$ (`item` / 10^i) % 10;
    `count[digit]++`;
▷ re-arrange items in `arr` according to `count`

- The radix sort algorithm is stable.
- The (hidden) final step requires a copy of the input array `arr`, so the radix sort algorithm is not in-place.

Radix sort: complexity analysis

Time complexity

- Suppose each item has at most d digits when written in base b (previous slides have $b = 10$.
- There are d calls to CountSortByDigit.
- Each digit has b possible values, so the complexity of CountSortByDigit is $\mathrm{O}(n + b)$.

Overall complexity: $\boxed{T(n) = \mathrm{O}(d \cdot (n + b))}$

Space complexity

- Final step in CountSortByDigit requires $\mathrm{O}(n)$.
- Auxiliary array `count` has length b.

Overall complexity: $\boxed{M(n) = \mathrm{O}(n + b)}$

Summary: complexities

Algorithm	Time (best)	Time (avg)	Time (worst)	Space
Bubble sort	$\mathrm{O}(n)$	$\mathrm{O}(n^2)$	$\mathrm{O}(n^2)$	$\mathrm{O}(1)$
Insertion sort	$\mathrm{O}(n)$	$\mathrm{O}(n^2)$	$\mathrm{O}(n^2)$	$\mathrm{O}(1)$
Selection sort	$\mathrm{O}(n^2)$	$\mathrm{O}(n^2)$	$\mathrm{O}(n^2)$	$\mathrm{O}(1)$
Merge sort	$\mathrm{O}(n\log(n))$	$\mathrm{O}(n\log(n))$	$\mathrm{O}(n\log(n))$	$\mathrm{O}(n)$
Quick sort	$\mathrm{O}(n\log(n))$	$\mathrm{O}(n\log(n))$	$\mathrm{O}(n^2)$*	$\mathrm{O}(\log(n))$
Counting sort	$\mathrm{O}(n+m)$	$\mathrm{O}(n+m)$	$\mathrm{O}(n+m)$	$\mathrm{O}(m)$
Radix sort	$\mathrm{O}(d\cdot(n+b))$	$\mathrm{O}(d\cdot(n+b))$	$\mathrm{O}(d\cdot(n+b))$	$\mathrm{O}(n+b)$

*This can be made extremely unlikely through reasonable pivot selection.

Summary: use cases

Algorithm	Stable	In-place	When to use
Bubble sort	Y	Y	Almost never
Insertion sort	Y	Y	Small or nearly sorted data
Selection sort	N	Y	When swaps are costly
Merge sort	Y	N	When stability is important, or need $\mathrm{O}(n \log(n))$ guarantee
Quick sort	N	Y	General-purpose sorting (fastest on average)
Counting sort	Y	N	Small range of (integer) values
Radix sort	Y	N	Large n, small digit range